

Identical Runs, Different Results: Benchmarking AI Coding Agents on Open-Weight Models

Eduardo Ariño de la Rubia and Szilard Pafka · September 2026

If your team deploys AI agents:

- **Do this.** Evaluate the agent and the model as one system, over several attempts. Reject the runs that broke your rules, rank the compliant ones on held-back data using the objective you actually care about, read the code of the leader, then measure that one model on a set you have never used.
- **Why.** On this task, repeated runs of one pairing produced materially different models, and they still did after the rule-breaking runs were removed. Three-run comparisons ranked pairings unreliably, and the most extreme scores were noncompliant.
- **Then.** Price the model you chose with its predictions at your own prevalence and costs, not with its AUC. Test again when the model, the agent version, the endpoint, the cache or the price changes.

Bottom line

Three different questions get confused when agents are compared, and this paper separates them: how good a compliant artifact is, how often an attempt produces one, and what a policy of several attempts plus selection delivers.

We report three studies on one task. The first is broad: six coding agents on six open-weight model endpoints, three runs each. The second is deep: three agents on two models, 52 runs each, 312 runs in all. The third is controlled: the same three agents on a larger model from the same family, 52 runs each, changing the model and nothing else.

Quality of compliant runs. In the deep study, the six pairing averages over compliant runs span 0.0095 AUC, while the median pairing varies by 0.0107 across its own compliant runs. The spread inside a pairing is larger than the spread between pairings, so three-run comparisons ranked them unreliably. That is a statement about the reliability of small comparisons, not evidence that the agents are equivalent.

Rate of compliant runs. 11 of 312 runs broke the task rules: six trained on data the rules put off limits, five computed features from the batch they were scoring. They are concentrated at the top of the score table. Removing them takes the best score in the study from 0.8293 to 0.7695, so the most impressive artifact was the least compliant one.

What a policy delivers. One attempt returns a compliant artifact 97 percent of the time, with a median holdout AUC of 0.7413. Three attempts returned one every time in our resampling, with a median of 0.7495, and ten attempts reach 0.7548. Repeat runs buy a better artifact cheaply; they do not buy a reliable ranking of vendors.

What a real upgrade is worth. Moving to the larger model in the same family gained 0.0097 AUC, seven standard errors from zero but only 0.93 times the run-to-run SD. The benchmark detects a genuine change cleanly; the agents are simply closer together than that. How much the upgrade bought also depended on the agent: 0.0152 for pi against 0.0055 for OpenCode, 2.8 times less from the identical change.

Running cost depended on the pairing too, by up to 23 times for the same job. We report it separately, after the three studies.

The task is machine learning; the lessons are about agents

We used a machine-learning task because it makes an agent's work measurable. The agent improves a model, and a hidden holdout scores the result. Nobody grades it by hand, the score has room to rise, and each run is cheap to repeat. Most work that companies automate gives no such score. The variation between runs still exists there, but nobody can see it.

This task exposes four risks that teams should test for in their own deployments: results vary between identical attempts; some attempts break the rules of the task; agents differ in how much effort they spend; and whether a prompt cache works depends on the agent and the endpoint together. We show that all four appear here. We have not established how common or how important they are in agentic work generally.

Most comparisons of AI agents ask which one is best. On a task where that question has a measurable answer, it was not the question that mattered.

Study 1: six agents, six model rows, three runs each

Run-to-run variation made three-run quality rankings unreliable

On four of six model rows, three runs per agent could not order the agents: the gap between the best and worst agent average is about the size of the gap between two runs of a single agent. This does not show that the agents perform equally. It shows that a three-run comparison cannot tell you which is better. On the other two rows the gap came from a single agent that spent little of its compute budget.

Table 1. Holdout AUC, three runs per agent on each model row.

Model row	Agent gap	Run-to-run gap	Ratio
DeepSeek V4 Flash, Ollama	0.021	0.021	1.0
DeepSeek V4.1 Flash, Ollama	0.019	0.018	1.0
Nemotron Super, Ollama	0.011	0.010	1.1
DeepSeek 4.1 Flash, LunaRoute	0.036	0.027	1.3
GLM-5.3 Flash, LunaRoute	0.022	0.012	1.9
GLM-5.3, LunaRoute	0.032	0.011	3.0

Quality and reliability are separate outcomes, so we report them separately. Of 108 scheduled runs on the six complete rows, three never started because the gateway rejected Codex’s request format, and two more delivered artifacts that broke the task rules. That leaves 103 scored. The three that never started are excluded from the AUC comparison, because there is no artifact to score, but they are a property of that deployed pairing: on DeepSeek 4.1 Flash through LunaRoute, Codex completed none of its three attempts.

Table 2. Study 1 run flow. The V4 Pro row was stopped at 85 percent of a weekly quota, four of its runs cut off mid-run; it appears only as a supporting observation about caching, in the cost section. 116 delivered files in total, 103 of them scored in the quality analysis.

Model rows	Scheduled	Started	Delivered an artifact	Compliant
Five complete rows	90	90	90	88
DeepSeek 4.1 Flash, LunaRoute	18	15	15	15
DeepSeek V4 Pro, Ollama (partial)	8	8	8	not assessed

Study 2: one pairing, fifty-two times

Three runs are enough to see that the variation exists. They are not enough to measure it, and not enough to rank anyone. So we took three open-source agents and the two fastest models of the first study and ran each pairing 52 times, holding the data, prompt, budget, machine type and parallelism fixed: 312 runs. The three agents had the highest three-run averages on the LunaRoute rows of Study 1, which is how we chose them rather than a finding: Study 1 could not rank them reliably.

Another benchmark averages the spread away

A Berkeley team published a related study while this one was running. HarnessTax (Pan and colleagues, September 2026, harnesstax.github.io) paired seven models with three agents — Claude Code, Codex CLI and pi — on SWE-bench Lite and Terminal-Bench 2.0. The designs differ where our main question lives. HarnessTax runs each pairing three times per task and averages the attempts, so it does not report how far repeated identical runs spread. That spread is what our second study measures, and it is wide enough to reorder agents. Their benchmarks also sit near their ceiling: the strongest model solves 97.8 percent of SWE-bench Lite attempts, which leaves a harness difference little room to show. They note the models may have seen those tasks in training. Our task is graded on a continuous score against a holdout the agent never sees, so differences have room to appear and rule-breaking stays detectable. HarnessTax’s cost finding agrees with ours, and we return to it after the three studies. The reliability problem we report is what you meet as soon as you try to measure any difference between agents.

A single run is a draw, not a result

A single run does not give you an agent’s average quality. It gives you one draw from its distribution. Among compliant runs, the six pairing averages span 0.0095 AUC while the median pairing varies by 0.0107 across its own runs. Two runs of the same pairing differ by 0.0147 AUC on average over all runs, and by 0.0285 at the ninth decile. One run delivered a model worse than the code it started from, and it had broken a task rule. Every compliant run beat the starting code, though the weakest by only 0.0007.

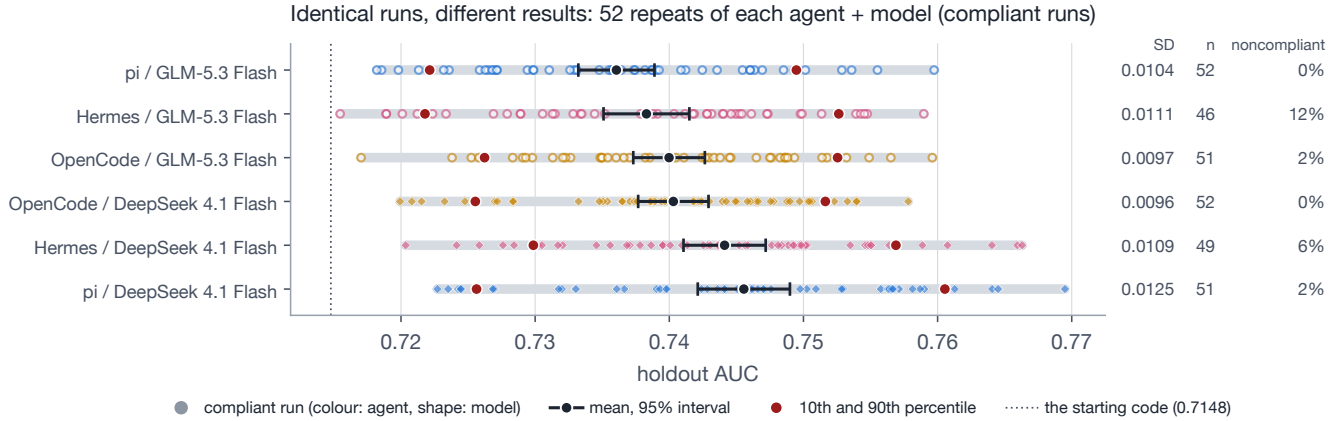


Figure 1. Compliant runs only: one mark per run, one row per pairing, in the order of the table below. Colour is the agent and shape the model, as in every figure here. Black dot and bar: the mean and its 95 percent interval. Red dots: the 10th and 90th percentiles, so the middle 80 percent of runs lie between them. Grey bar: the full range. Dotted line: the starting code. At right: the SD and number of compliant runs, and the share of the pairing’s 52 runs that broke a task rule. Those 11 runs are left out of the plot; they scored 0.7116 to 0.8293 and are described below.

Table 3. Compliant runs only, except the last column. The starting code scores 0.7148 AUC and 0.7041 average precision. Removing the 11 noncompliant runs lowers the study’s best score from 0.8293 to 0.7695 and the median pairing SD from 0.0133 to 0.0107.

Agent and model	Compliant runs	Mean	Median	SD	95% interval of the mean	Best	All-run mean
pi, GLM-5.3 Flash	52	0.7361	0.7359	0.0104	0.7332 – 0.7389	0.7597	0.7361
Hermes, GLM-5.3 Flash	46	0.7383	0.7408	0.0111	0.7351 – 0.7415	0.7590	0.7427
OpenCode, GLM-5.3 Flash	51	0.7400	0.7402	0.0097	0.7373 – 0.7427	0.7596	0.7417
OpenCode, DeepSeek 4.1 Flash	52	0.7403	0.7418	0.0096	0.7377 – 0.7429	0.7578	0.7403
Hermes, DeepSeek 4.1 Flash	49	0.7441	0.7437	0.0109	0.7411 – 0.7472	0.7663	0.7459
pi, DeepSeek 4.1 Flash	51	0.7456	0.7462	0.0125	0.7421 – 0.7490	0.7695	0.7455

Three runs cannot tell you which agent is better

We drew three compliant runs at random from each pairing, 20,000 times, and compared pairing averages within a model. The weaker pairing won 28 to 43 percent of those draws, and the two closest comparisons are near a coin toss. A three-run report can state either order.

How many runs it would take depends on which difference you want to detect, and the answer is not portable. Under the conventional approximation, detecting the widest gap among these six pairings, 0.0095, would take about 20 runs of each. But that contrast changes both the agent and the model. The widest gap between two agents *on the same model* is 0.0053, and detecting that would take about 66 runs of each. Smaller differences need more. These are also differences we chose after seeing the data, which flatters them; a comparison fixed in advance should be sized on the difference that would change your decision, not on the largest one observed.

Which agent looks best depends on the model

The six pairings are three agents crossed with two models, so their means can be read both ways. Across agents, the three sit within 0.0039 AUC of each other on GLM-5.3 Flash, with OpenCode highest and pi lowest, and within

0.0053 on DeepSeek 4.1 Flash, where the order reverses: pi highest, OpenCode lowest. Across models, moving from GLM-5.3 Flash to DeepSeek 4.1 Flash raised pi by 0.0095, Hermes by 0.0058 and OpenCode by 0.0003. pi's gain exceeds OpenCode's by 0.0092, 3.1 standard errors of that difference.

Study 2: the same means two ways (colour: agent; shape: model; compliant runs)

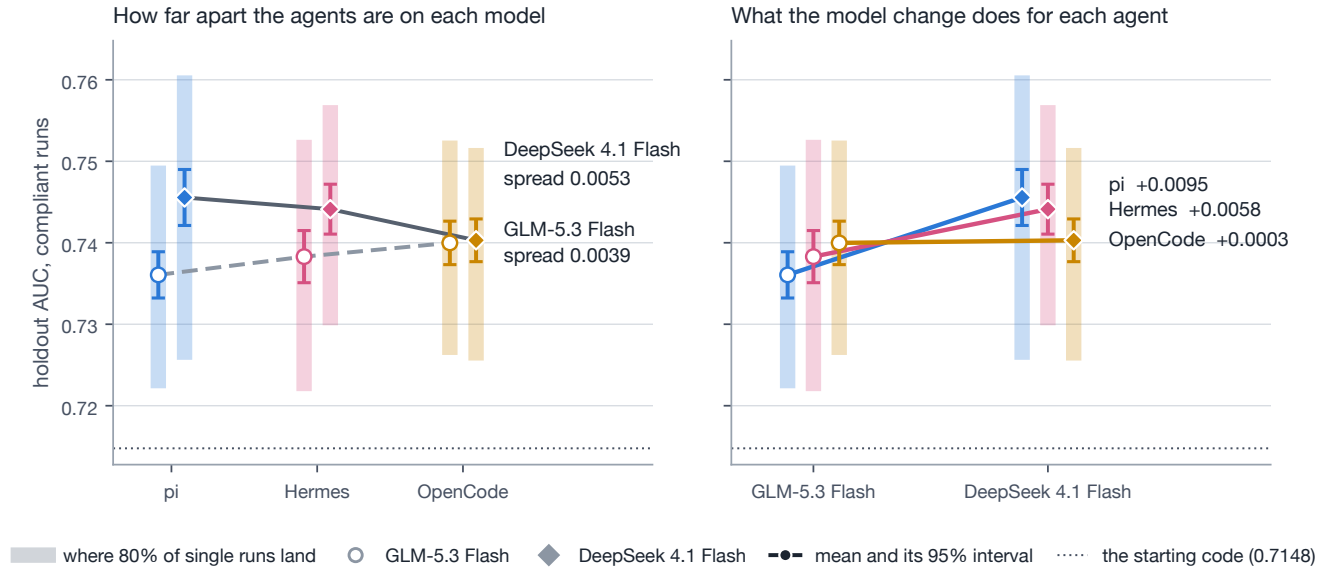


Figure 2. The six Study 2 means read two ways, compliant runs. Left: agents on the axis, one line per model, labelled with how far apart the agents are on it. Right: models on the axis, one line per agent, labelled with its change. Colour is the agent and shape the model. Thick bar: the 95 percent interval of the mean. Pale band: where the middle 80 percent of single runs land. The two measure different things, and overlapping bars are not a test; the text gives the tests.

Two cautions. Neither end of the reversal is strong on its own: pi trails OpenCode by 2.0 standard errors on GLM-5.3 Flash and leads it by 2.4 on DeepSeek 4.1 Flash. And we noticed the pattern after looking at the data, across two models from different families, so it is a pattern to test, not a finding. The pale bands make the section's larger point again: each of the six means lies inside every pairing's middle 80 percent of runs, so no single run could show any of this. A ranking of agents is a claim about agents on one model.

The tails are method choices, and some of them break the rules

The agents do not merely jitter. They sometimes take a different approach, and a few of those approaches are not allowed by the task. Six of the 312 runs added the labelled evaluation file to their training data; five of those were Hermes on GLM-5.3 Flash, and the six scored 0.7215 to 0.8293. Five other runs computed features from the batch they were asked to score, which the task also forbids, and scored 0.7116 to 0.8036.

Both are rule violations rather than contamination of the hidden test. The holdout stayed hidden in every case: no run read it, and every score comes from re-running the delivered code against it. A run that trained on the evaluation file had more labelled data than the rules allow, from the same year as the holdout, so its score may be a real gain obtained with unauthorized data rather than an illusion. A run that computes features from the scoring batch depends on batch composition at prediction time, which this task forbids; whether that is deployable in your own system depends on whether the batch is available when you predict.

This is why selection needs an audit. The noncompliant runs sit at the top of the score ranking, so the run you would pick on score alone is the run most likely to have broken a rule. Picking the maximum of 52 scores on one holdout also turns that holdout into a selection set, and the winner's margin is then part real and part luck.

Detection is the weakest remedy. Two checks find these after the fact: search the delivered code for evaluation labels reaching training data, and trace whether any feature is computed from the frame handed to the prediction function. Prevention is better. Put the evaluation labels behind a scoring interface, so unauthorized training is impossible rather than detectable. Test whether a row's prediction changes when the batch around it changes, which catches

batch-dependent features without reading any code.

What several attempts buy

The advice above is to attempt more than once, reject the noncompliant runs and keep the best of the rest. That policy can be measured. Resampling attempts from the 312 observed runs, 20,000 draws per number of attempts:

Table 4. Best-of-k over compliant runs, pooled across the six pairings. Selection here is on holdout AUC, the benchmark’s own metric, so these are an upper bound on what score-based selection achieves; the procedure we recommend, selecting on a decision objective and confirming on untouched data, would give a lower number.

Attempts	At least one compliant	Median kept	5th percentile	95th percentile
1	96.6%	0.7413	0.7220	0.7589
3	100%	0.7495	0.7355	0.7641
5	100%	0.7526	0.7412	0.7645
10	100%	0.7548	0.7463	0.7663

Three attempts move the median artifact 0.0082 AUC above one attempt, and ten attempts 0.0135. The returns fall away quickly, and the floor rises faster than the ceiling: from one attempt to ten, the 5th percentile improves by 0.0243 and the 95th by 0.0074. Attempts mostly buy protection against a bad draw. At the token prices of this study that protection costs roughly \$2 for three attempts and \$8 for ten, before the audit work, which is the real cost.

That is the sense in which choosing one good artifact is cheaper than ranking two vendors: three to ten attempts against roughly 66 runs of each pairing.

Budget use predicts score inside a pairing

In both studies, the runs that spent more of their measured compute scored higher. The broad study found a rank correlation of +0.59 across 103 runs, where it could have meant that some agents simply work harder than others. The deep study holds the agent and the model fixed: within a pairing the correlation is **+0.55** over 312 runs, with a 95 percent interval of +0.47 to +0.63, and +0.56 over compliant runs alone. Every pairing points the same way, from +0.38 to +0.70.

Budget use against score inside each pairing (valid runs; dotted line: the starting code)

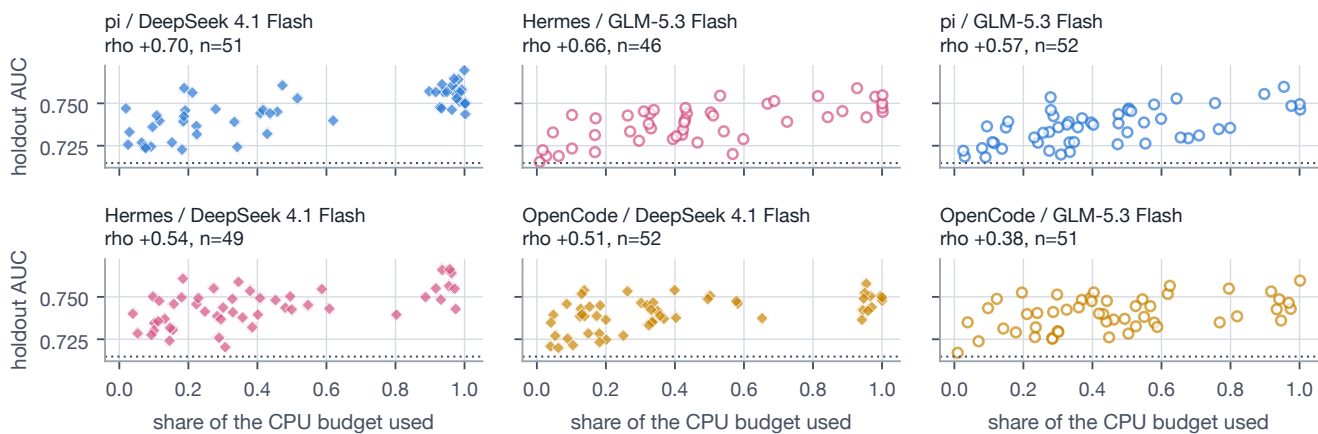


Figure 3. Budget use against score inside each pairing, compliant runs only. Colour is the agent and shape the model. Dotted line: the starting code.

What the meter shows is CPU seconds consumed by Python inside the container. It is not a measure of search effort. Every one of the 95 runs that spent under a quarter of the budget still ran all 40 of its counted experiments: they did not stop early, they used cheaper methods. Nor does the meter see the model’s own generation, or reasoning the

agent does in context. A low reading is a fact about resource use and a reason to read the run’s experiment trace, not evidence of carelessness.

The association is weaker and less precisely estimated among high-budget runs: +0.43 among compliant runs above a quarter of the budget, +0.29 above half, and +0.11 above three quarters with an interval spanning zero. Narrowing the sample also narrows the predictor and shrinks the sample, so this is not by itself a demonstration of diminishing returns.

Study 3: the same agents, a larger model

The larger model’s gain is about the size of run-to-run noise

A third study changed one thing and measured what it bought. GLM-5.3 is the larger model in the family whose Flash version Study 2 used. We ran the same three agents against it on the same 52 seeds, with the same data, prompt, budget, machine type and parallelism: 156 runs in which only the model differed.

The larger model scored **0.0097 AUC above Flash**, averaged over the three agents, with a 95 percent interval of 0.0071 to 0.0124. That is 7.2 standard errors from zero, so the difference is real. It is also **0.93 times the run-to-run SD of a single pairing**. Run each model once, and the smaller one still comes out ahead 28 percent of the time: 15 percent with pi, 35 with OpenCode.

This answers a fair question about the first two studies. When a benchmark reports that agents do not separate, a reader should ask whether it can detect anything at all. It can. One genuine change moved the score by seven standard errors on the same design that could not rank three agents. The agents are close together; the measurement is not blunt.

Table 5. Holdout AUC by agent for the two models, compliant runs only, with the same two screens applied to both.

Agent	GLM-5.3	Runs	GLM-5.3 Flash	Runs	Gain	95% interval
pi	0.7513	47	0.7361	52	+0.0152	+0.0110 – +0.0193
Hermes	0.7469	48	0.7383	46	+0.0086	+0.0033 – +0.0142
OpenCode	0.7455	50	0.7400	51	+0.0055	+0.0016 – +0.0094
All three	0.7480	145	0.7383	149	+0.0097	+0.0071 – +0.0124

The larger model wins on average, but one run of each still favours the smaller model 28% of the time

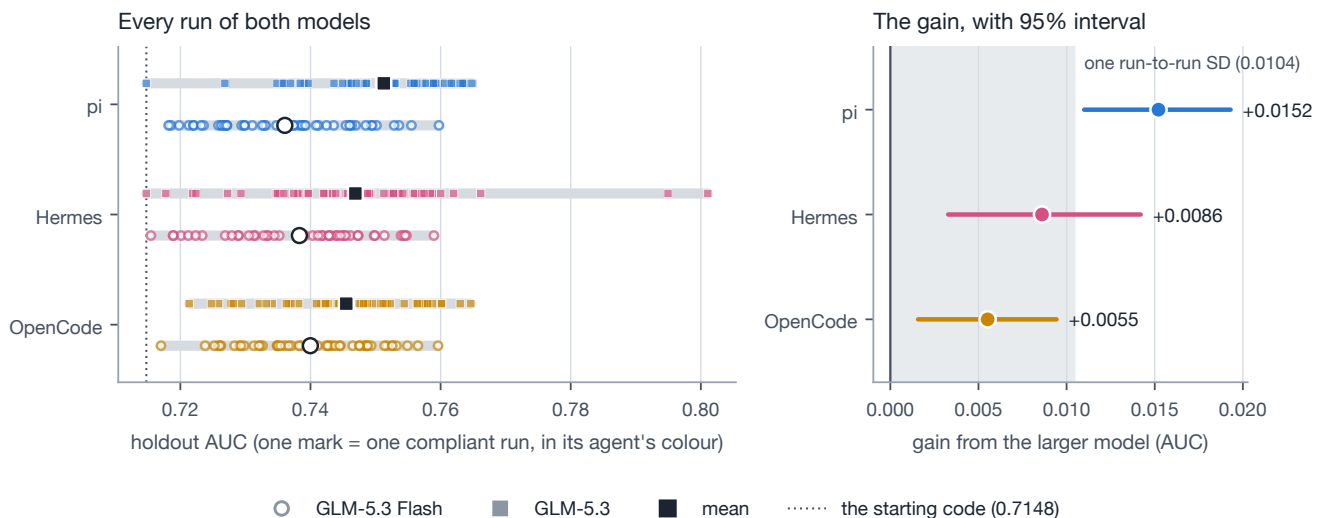


Figure 4. Left: every compliant run of both models, three agents, coloured by agent; hollow circles are GLM-5.3 Flash and squares GLM-5.3. Black marks are the averages; the dotted line is the starting code. Right: the gain from the larger model with its 95 percent interval, against a shaded band one run-to-run SD wide.

How much the upgrade buys depends on the agent it is paired with. pi gained 0.0152 and OpenCode gained 0.0055, 2.8 times less, and that difference clears zero at 3.3 standard errors. Hermes falls between them and separates from neither, so the comparison we can defend is pi against OpenCode, not a ranking of three. The same model upgrade, on the same task under the same budget, was worth nearly three times as much in one agent as in another. Buying a better model is not a decision about the model alone.

Study 3: the same means two ways (colour: agent; shape: model; compliant runs)

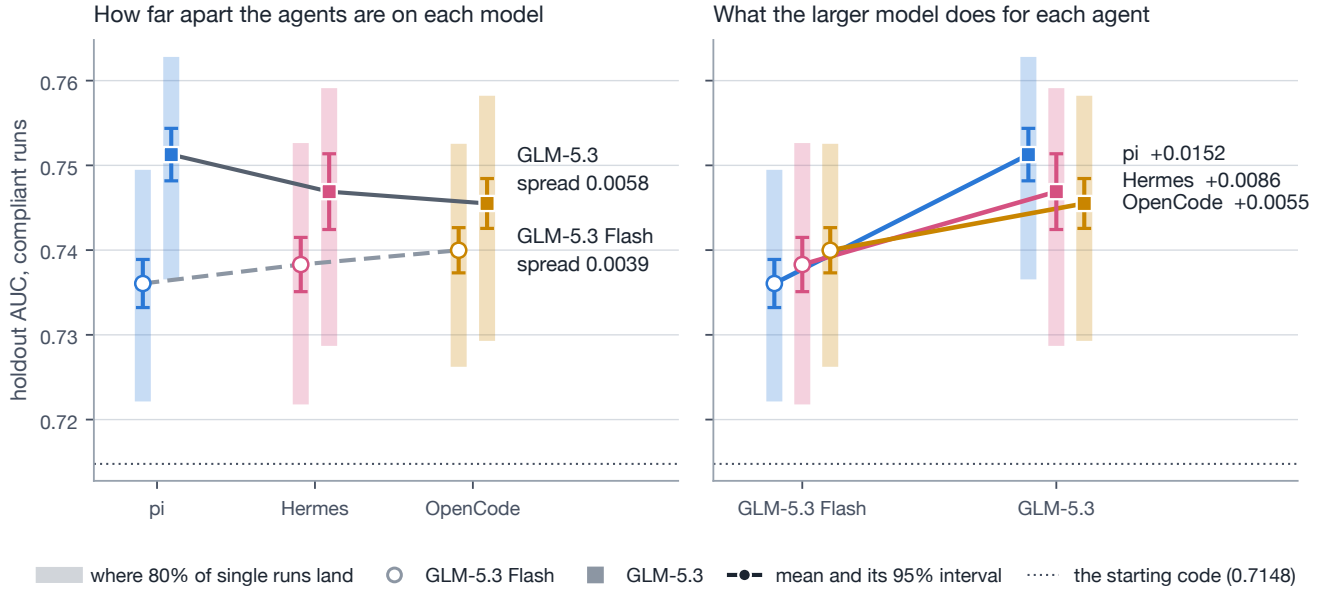


Figure 5. The Study 3 means read the same two ways as in Study 2, on the same scale. Left: agents on the axis, one line per model. Right: models on the axis, one line per agent, labelled with its gain. Colour is the agent and shape the model; thick bar: the 95 percent interval of the mean; pale band: the middle 80 percent of single runs.

Study 2 showed the same ordering, but the two are less independent than they look. There too, pi gained most from the stronger model and OpenCode least. Both comparisons start from the same GLM-5.3 Flash runs, on which pi trails OpenCode by 0.0039, and that gap counts toward pi's larger gain in both studies. What the two studies add separately is that pi leads OpenCode on each stronger model, by 0.0053 on DeepSeek 4.1 Flash and 0.0058 on GLM-5.3, at 2.4 and 2.6 standard errors. The pale bands carry this study's point as well: pi's gain is 7.1 standard errors from zero in the means, yet pi's runs on the two models overlap across the middle of both bands.

What the study held fixed. Neither arm set a reasoning level. Both GLM models document the same default, thinking enabled at the top effort setting, and a probe of the endpoint confirmed it: with no setting sent, a short prompt spends about as much thinking as the maximum and roughly ten times what the lowest setting spends. The arms differ in model scale, not in how hard the model was asked to think. Runs were spread across four identical machines, and the box effect was negligible ($F = 1.27$). Scores did not drift: the first and second halves of the arm average the same to four decimals. Eight of the 156 runs broke a task rule and are excluded above, under the screens Study 2 uses.

Putting the three effects on one scale gives the resolving power of this benchmark, in units of the run-to-run SD and in the number of runs each effect would need:

Table 6. Effect sizes against the median within-pairing SD of 0.0104, with the run count each would need under the approximation used above. The first row compares against a fixed number rather than a second noisy arm, so its count is a lower bound.

Effect	Size	ln SDs	Runs per arm
starting code to what an agent delivers	+0.0281	2.7	2
GLM-5.3 Flash to GLM-5.3	+0.0097	0.9	18
widest gap between two agents, on GLM-5.3	+0.0058	0.6	52
widest gap between two agents, on GLM-5.3 Flash	+0.0039	0.4	113

Three runs answer one question reliably: did the agent improve on the code it started from. The larger model’s gain takes about 18. Telling two agents apart on a fixed model takes 52 to 113. Most published comparisons are made at three.

What a difference is worth

Price it with predictions, not with AUC

Do not convert an AUC gap into money. AUC summarises the ranking across every threshold, so a 0.05 gap does not tell you what changes at the one threshold you use. Score the candidate models on held-back data, take their predictions, apply the decision rule you actually run, and count true positives, false positives and false negatives at your own prevalence and costs.

Here is that calculation on two of our own models, the best compliant run of a pairing and its median run, 0.7695 and 0.7471 AUC. Suppose one million decisions a month, one in a hundred of them a true positive, a team that can review a fixed number of cases, \$10 for a false alarm and \$100 for a miss. We assume that only the prevalence differs from the benchmark and that the score distribution within each class is unchanged; under that assumption the true-positive and false-positive rates measured here carry over, and the counts below follow.

Table 7. Illustrative, not a forecast. Both models were selected and scored on the same holdout, with no second untouched evaluation, so these differences are not validated prospective savings. The point is the crossing: the better model by AUC is worse at the tightest capacity.

Cases reviewed each month	Caught by the 0.7695 model	Caught by the 0.7471 model	Difference
5,000	562	580	–\$1,920
20,000	1,351	1,279	\$7,955
100,000	3,714	3,421	\$32,156

That crossing is also an instruction. Rank your candidates on the objective you will actually use, on a selection set, and then measure the one you chose on untouched data. Selecting on AUC and pricing afterwards can hand you the artifact that loses money at your operating point.

Read our average precision figures the same way. AP depends on how common the positive class is, and this holdout was built with the two classes in equal numbers, while real departure delays are far rarer. So these AP values compare runs on this benchmark fairly and are not the precision you would see in production. You cannot rescale them either: true-positive and false-positive rates carry across to a different prevalence under the assumption above, precision does not, so recompute it from the predictions at your own rate.

What it costs

Cost is a separate outcome from quality, and it also depended on the pairing.

Cost moved with the pairing

In Study 1, on DeepSeek V4 Flash, rate-card cost followed the prompt cache. The endpoint served 92 to 98 percent of pi’s and Codex’s input from cache, and 2 to 11 percent of Claude Code’s. Two effects combined. Repricing Claude Code’s own token volume at pi’s hit rate gives \$0.20 a run, against its actual \$1.86. So the cache accounts for about nine times, and Claude Code’s larger token volume for about two and a half.

Table 8. Cached share of input and rate-card cost per run, DeepSeek V4 Flash, Ollama Cloud. No money changed hands: a flat-rate subscription paid for the runs, and every cost is a rate-card projection.

Agent	Run 1	Run 2	Run 3	Mean cost
pi	98%, \$0.08	98%, \$0.11	97%, \$0.05	\$0.08
OpenClaw	no record	98%, \$0.11	97%, \$0.07	\$0.09
Codex	96%, \$0.14	96%, \$0.19	92%, \$0.13	\$0.15
OpenCode	80%, \$0.24	64%, \$0.19	66%, \$0.49	\$0.31
Hermes	56%, \$1.09	89%, \$0.47	74%, \$0.24	\$0.60
Claude Code	11%, \$1.41	9%, \$2.95	2%, \$1.21	\$1.86

The cache failure belonged to the pairing, not to either half. Claude Code cached 96 to 97 percent of its input on DeepSeek V4.1 Flash and 73 to 84 percent on Nemotron Super. Every agent on those two models cached 70 to 99 percent. The failure did repeat on one more model, DeepSeek V4 Pro, whose row the quota stopped after eight runs: Claude Code cached 4 and 11 percent while the other five agents cached 83 to 99 percent. The quota cut off both Claude Code runs, but it also cut off both of pi's, which still cached 91 and 98 percent. On V4 Flash, Claude Code's own request log shows the cache returning almost nothing from the first request, at 14,000 tokens, and never recovering, while on V4.1 Flash the same agent reads its prompt back at up to 122,000 tokens. So context size does not explain it. The cause is open. A test with byte-identical prompts through both endpoints would separate the client from the server.

Rate-card cost also moved between identical runs. In Study 1, Hermes cost \$0.24, \$0.47 and \$1.09 for the same work on the same pairing, a swing of 4.6 times. Across the 25 pairings with three complete costs, the median swing was 2.2 times.

Another group found the same cost gap

HarnessTax, the Berkeley study described in Study 2, reached the same cost finding on different tasks. Agent choice barely moved task success there, but it moved cost by as much as five times, and Claude Code cost about twice what pi cost on the same model. Two studies with different tasks, different models and a different metric reached the same conclusion, which makes it firmer than either study alone: the cost gap is real and shows up on public coding benchmarks.

What the repeats cost

The 312 runs of Study 2 read 1.86 billion input tokens and wrote 25.1 million, a ratio of about 74 to 1, because the agent re-reads a growing transcript at every step. Flat-rate subscriptions paid for them. Repricing that same token ledger at OpenRouter list prices on 16 September 2026 gives about \$244 on the two models we ran.

A third of what these agents wrote was thinking nobody asked for. Of the 25.1 million written tokens, 8.7 million are reasoning tokens. We never set a reasoning level: the configuration sends no reasoning_effort, no thinking budget, and one agent declares that it does not support the field at all. Omitting it is not a neutral choice. Both GLM models document max, the top of their scale, as the default, and a probe of the endpoint confirms it: with no field set, a short prompt spends about as much thinking as max and roughly ten times what low spends. Our own token accounting missed all of it, because every one of these agents reports reasoning separately from output and we recorded only the output. Check whether yours does the same before you trust a bill or a budget.

Table 9. Hypothetical repricing of the observed token ledger, reasoning tokens included and billed at the output rate. This is what our tokens would have cost at other prices, not what those models would cost to do this task: another model would produce a different transcript length, completion rate and caching profile, in either direction. Cached-input rates for the last column are listed in appendix D. Token cost is also not the whole bill, which includes the machines and the audit work.

Model	Price per M tokens, in / out	Repriced ledger	If 90 percent of input were cached
GLM-5.3 Flash and DeepSeek 4.1 Flash, as run	\$0.10 / \$0.33 and \$0.15 / \$0.60	\$244	\$54
GLM-5.3	\$1.40 / \$4.40	\$2,717	\$807
Gemini 3.1 Pro	\$2.00 / \$12.00	\$4,026	\$1,009
Kimi K3	\$2.65 / \$13.28	\$5,265	\$1,334
Claude Opus 5	\$5.00 / \$25.00	\$9,939	\$2,397
GPT-5.5 Pro	\$30.00 / \$180.00	\$60,385	\$10,106

The pairing drives both the variation and the bill. Hermes read 8.65 million input tokens per run and pi read 3.93 million, for the same task, the same budget, and compliant averages within 0.006 of each other. Caching then matters more than the rate card, and on the Study 1 pairing above the hit rate collapsed to 2 percent.

What to do

- **Evaluate the pairing, not the parts.** Testing an agent on one model, or a model through one agent, can badly mislead you about the combination you will deploy. Record whether attempts started and completed, not only how the finished ones scored.

- **Attempt the job several times.** One attempt is a draw. On this task three attempts moved the median artifact by 0.008 AUC and raised the worst case by more; ten attempts added little beyond three.
- **Reject first, then rank.** Check every attempt against your rules before you compare scores. Better, make the violation impossible: put evaluation labels behind a scoring interface, and test whether predictions change with batch composition.
- **Rank on your objective, then confirm on untouched data.** Rank compliant candidates on a selection set using the decision rule you will run, read the leader's code, then measure that one model once on data you have never used. The set you select on always flatters the winner.
- **Read the experiment trace of any run that used little compute.** It used cheaper methods, which may or may not be what you want. It is not necessarily a run that gave up.
- **Instrument the parts nobody looks at.** Record the cached share of input and the compute used on every run. A pairing whose cache fails can cost about nine times more on the cache alone.
- **Price the winner with its predictions,** at your prevalence and your costs, and price the failures too: one stalled run still cost \$1.21 at rate-card prices and delivered a weaker result.

Appendix A: design

This benchmark builds on our earlier study, published in April 2026, which showed that AI coding agents can automate XGBoost feature engineering and hyperparameter tuning ([xgboost-autoresearch](#)).

Each agent was given a model, a dataset, a written task and a compute budget, and asked to improve an XGBoost classifier by editing one file.

	Study 1	Study 2
Question	does the pairing matter?	how much does one run vary?
Agents	Claude Code, Codex, pi, OpenCode, Hermes, OpenClaw	pi, OpenCode, Hermes
Model rows	six, on Ollama Cloud and LunaRoute	GLM-5.3 Flash and DeepSeek 4.1 Flash, LunaRoute
Runs	3 per pairing: 108 scheduled on the six complete rows, plus an 8-run partial row; 116 delivered files; 103 scored	52 per pairing; 312, all scored
Dates	13 to 15 September 2026	15 to 16 September 2026
Machines	4 cores, 8 GB per run	four identical 16-core machines, 4 runs each

Both studies share the task: airline departure delay, binary classification, scored by AUC on a hidden holdout of 1,000,000 rows from 2006; training 100,000 rows from 2005 and evaluation 100,000 rows from 2006. The split is fixed and identical for every run. Every run had the same prompt, task files, time limits, compute budget and scoring. Web search was off by instruction. Counted experiments were 40 per run, each a call to the provided run script; 244 of Study 2’s 312 runs used all 40, and the median run used 40.

The holdout has one million rows, so the standard error of an AUC on it is about 0.0005. Differences of 0.01 to 0.04 between runs are not measurement error. The holdout’s two classes were built in equal numbers, which is not the rate at which flights are late, so AP figures rank runs here and do not predict production precision.

In Study 2 the seed number is only a label: nothing in the code reads it, and the starting code always uses the same random seed, so the differences come from the agent. Every machine ran all six pairings, so the machine and the pairing cannot be confused; the machine did not change the scores.

Exclusions differ between the studies, on purpose. Study 1 asks which pairing is better, so five runs are excluded from its AUC tables: three Codex runs that never started because the gateway rejected the request format, one that concatenated the evaluation rows into its training data, and one whose delivered code computed count features on the data being scored. The three that never started still appear in the run-flow table, because failing to start is a property of the pairing. Study 2 asks what the distribution looks like, so every run is counted, compliant results are reported separately from all-run results, and the figures plot compliant runs only.

Reproduction. The data repository, github.com/earino/identical-runs-different-results, holds the task rules and prompts, agent versions and configurations, model and endpoint identifiers, one row per run for all three studies, and the holdout. For Studies 2 and 3 it adds every version of the code each agent delivered, the audit records, and the scoring, statistics and figure code. Its VERIFY.md maps each headline claim to the command that reproduces it. The full run trees, 43 GB of logs and container state, stay out of it.

Built with Claude. Claude, working in Claude Code, helped build the benchmark, operate the runs and draft this paper. Claude Code is also one of the six agents tested in Study 1. Every agent was scored the same way, on the same hidden holdout.

Appendix B: the compute budget

Each run had 18,000 CPU seconds of Python compute, with a container-level stop at 46,000. The budget was calibrated by re-running earlier delivered code on the same hardware.

What the meter counts. Every Python process started inside the container, summed over user and system CPU time, including child processes. A hook in the interpreter records each process on exit. Every model fit is logged separately with its row count and duration.

What it does not count. Time the model spends generating tokens, and reasoning the agent does in context rather than in code. An agent that plans carefully and fits small models meters as a low user of the budget. Budget use is therefore a measure of compute invested in fitting and nothing more: in Study 2, all 95 runs that used under a quarter of the budget nonetheless ran all 40 counted experiments.

Enforcement. The hook refuses to start a new process once the budget is spent, and a watchdog stops a running process that crosses it. Because the watchdog checks every ten seconds, two runs overshoot, by at most 17 seconds. No run was stopped by the container-level cap.

Appendix C: statistics

Agent spreads per row, Study 1. The agent gap is the best agent’s three-run average minus the worst agent’s. The run-to-run gap is the median, across a row’s agents, of each agent’s best run minus its worst. Six agents with no real difference would produce a ratio of about 0.9 by chance under a normal model. For two agents chosen in advance, that design separates a difference of about 0.02.

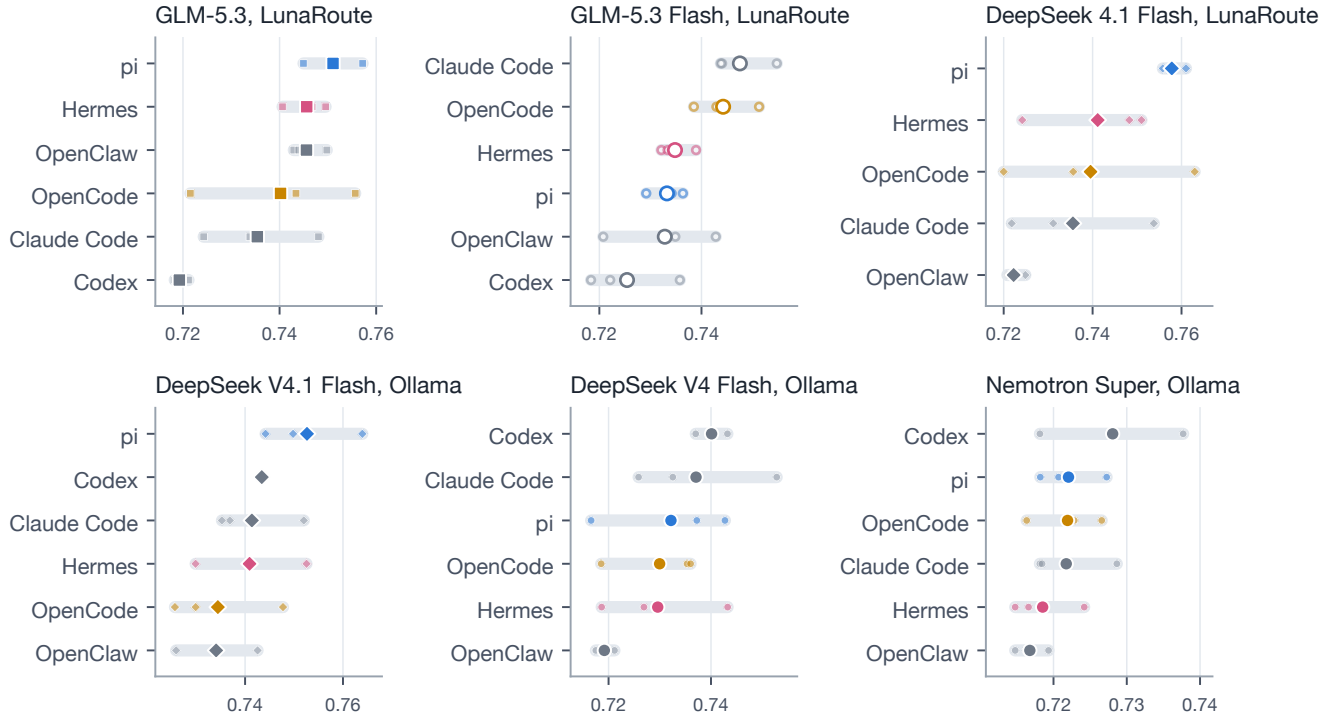


Figure 6. Holdout AUC, one panel per model row of Study 1. Per agent: the mean of three runs (large mark), the runs (small marks) and their range (bar). pi, Hermes and OpenCode, the agents Studies 2 and 3 follow, keep their colours; the other three are grey.

Three-run draws, Study 2. For each pairing we drew three compliant runs at random 20,000 times and compared pairing averages within a model; the weaker pairing won 28 to 43 percent of draws. Sample sizes use $16 \text{ sd}^2 / \text{gap}^2$, the usual approximation for 80 percent power at a 5 percent two-sided test, with the median compliant-run SD of 0.0107: about 20 per pairing for the widest gap among all six pairings (0.0095, which changes agent and model together) and about 66 for the widest gap between two agents on one model (0.0053).

Compute against score. Study 1: rank correlation +0.59 over 103 runs, bootstrap interval +0.45 to +0.71, permutation p below 0.001. Study 2, within pairings (runs ranked inside their own pairing, then pooled, so pairing-level differences cannot drive it):

Subset	Runs	Rank correlation	95% interval	p
All runs	312	+0.55	+0.47 to +0.63	below 0.001
Compliant runs	301	+0.56	+0.47 to +0.63	below 0.001
Compliant, above a quarter of budget	208	+0.43	+0.31 to +0.54	below 0.001
Compliant, above half	110	+0.29	+0.12 to +0.45	0.001
Compliant, above three quarters	70	+0.11	−0.15 to +0.37	0.402

Intervals are bootstrap over runs with 2,000 resamples; p-values come from permutation, shuffling scores within each pairing. The subsets count compliant runs, which is why they sum against 301 rather than 312: 93 compliant runs used a quarter of the budget or less, and 95 of all 312 did.

An alternative measure of effort. Fits predict the score far more weakly than CPU seconds: +0.25 over Study 1's runs and +0.09 within Study 2's pairings. The number of fits says little; the compute those fits consume says more.

Best-of-k. Attempts are drawn with replacement from a pairing's 52 observed runs, 20,000 draws per number of attempts, pooled across pairings. An attempt counts as acceptable if it is compliant; selection is on holdout AUC.

Causal reading. The compute relationship is correlational and its direction is not established. An agent whose search is going well may continue, which would produce the same pattern.

Conventions. Standard deviations are sample standard deviations, reported to four decimal places.

Appendix D: money, tokens and audits

No metered spend occurred. Flat-rate plans paid for these runs: an Ollama Cloud subscription and flat-rate access to LunaRoute as a tester. Study 1 costs apply Ollama Cloud's published rate card from 2026-09-11 to the tokens each run reported. Study 2's repricing applies OpenRouter's published prices from 2026-09-16. Neither plan issues an invoice to reconcile against. The one external check available is that modelled spend tracked Ollama Cloud's own weekly quota meter to within about two percent at two separate points in the same week.

Cached-input prices used in the last column of the repricing table, per million tokens: GLM-5.3 Flash \$0.02, DeepSeek 4.1 Flash \$0.003, GLM-5.3 \$0.26, Gemini 3.1 Pro \$0.20, Kimi K3 \$0.303, Claude Opus 5 \$0.50. GPT-5.5 Pro lists no cached-input rate, so its column charges only the uncached tenth of input, which flatters it. The column assumes nine tenths of input served from cache and output priced in full.

The 23 times figure. It is the ratio of Claude Code's three-run mean to pi's three-run mean on DeepSeek V4 Flash. Computed from unrounded costs it is 22.5, and 23 when rounded. Two of Claude Code's three costs come from its final usage report and one, for the stalled run, from its request log.

Per-request validation. Claude Code keeps a log of every request it sends. Rebuilt from that log, its usage reproduces the cost of all 15 runs that reported one, to within ten cents. Two cautions apply. Each message is repeated in the log whenever the conversation is saved again, so entries must be deduplicated by message id. And the cache-creation field reads zero on every request, so a zero there means nothing.

Audits. Every run is scored by re-running its delivered code against the holdout, which no agent sees. Three checks run over the delivered files. A screen flags any run whose reported evaluation score exceeds its holdout score by more than 0.03. A leak audit traces whether evaluation data reaches a fit call. A third check, added for Study 2, traces statistics computed on the frame handed to the prediction function.

Across Study 1's 116 delivered files, one run had concatenated the evaluation rows into training and the screen had already caught it; the screen also caught the run whose count features were computed on the data being scored. Three runs used the evaluation set to stop training early, which the task rules permit. Applying Study 2's third check to those files in hindsight gives one real case, the run Study 1 had already excluded.

Study 2 ran all three checks over 312 delivered files. The leak audit found 32 code-level hits, of which 26 are the permitted early-stopping use, with a largest evaluation-versus-holdout gap of 0.0018. The third check is what the screen misses: of the five real cases, only one had a gap large enough to trigger the screen.

Screens are screens. The frame-statistics check over-flags: it raised two runs that a person cleared, one that grouped rows only to index them and one that fitted a label-taking encoder which never sees a scored frame. Every hit in both studies was read before it was counted.

Appendix E: the words we use

AUC. The chance that the model scores a true positive above a true negative, taken across every threshold. It measures ranking, not the decisions you make at one cut-off.

Average precision (AP). How clean the top of the ranked list is. It depends on how common the positive class is, so an AP measured on a benchmark does not transfer to a different prevalence.

Prevalence. How common the positive class is in your data. Our holdout was built with the two classes in equal numbers; real departure delays are far rarer.

Operating point. The threshold you act on, such as the number of cases your team can review in a month. Two models can rank the same overall and behave differently here.

Held-back data. Rows the agent never saw, kept for scoring. Rank candidates on one set, then measure the model you chose on a second one you have never used.

Compliant run. A run whose delivered code obeys the task rules: it trains only on the training file, and computes no feature from the batch it is scoring.

Budget share. The fraction of a run's 18,000 CPU seconds that its Python processes actually consumed.